

24-ma'ruza. Strukturalar

Reja:

1. Struktura haqida
2. Strukturalarni e'lon qilish
3. Ro'yxatlar

Strukturalar - eng qiziqarli tur hisoblanadi, chunki ular oddiy tur sifatida ham (*new operator* bilan ishga tushirilmasdan) ham, mos yozuvlar turi sifatida ham ishlashi mumkin. Struktura-bu bitta nom ostida guruhlangan ma'lumotlar turlari to'plamidir.

Sinflar ma'lumotlarning berilganlar turlari hisoblanadi. Bu shuni anglatadiki, to'g'ridan to'g'ri kirish mumkin bo'lgan oddiy turdagi qiymatlardan farqli o'laroq, ma'lum bir sinf obyektlariga mos yozuvlar orqali kirish mumkin. Ammo ba'zida oddiy turdagi qiymatlar sifatida obyektlarga to'g'ridan to'g'ri kirish imkoni bo'lishi foydali bo'ladi. Ushbu muammolarni hal qilish uchun C# sinfga o'xshash strukturani taqdim etadi, ammo ma'lumotlarning berilganlar turi emas, balki qiymat turi hisoblanadi.

Strukturalar *struct* kalit so'zi yordamida e'lon qilinadi va sintaktik jihatdan sinflarga o'xshashdir. Quyida strukturani e'lon qilishning umumiy shakli keltirilgan:

```
struct nom
{
    //Struktura tanasi
}
```

Bu yerda, nom - strukturaning nomini bildiradi.

Aytaylik, biz shaxsni dasturiy ta'riflashimiz kerak. U quyidagi parametrlar bilan tavsiflanadi: *ism*, *familiya*, *yosh*. Ushbu ma'lumotlarni saqlash uchun uchta o'zgaruvchi yaratilishi mumkin va bizda bitta odam bo'lsa, bu oson bo'ladi, lekin agar bizda ulardan yuztasi bo'lsa qanday bo'ladi? Ko'p odamlarni tavsiflash kerak bo'lganda, uchta massivni yaratish mumkin, ularning har birida shaxsning bitta atributi - *ism*, *familiya* va *yosh* mavjud. Agar o'nta xossa mavjud bo'lsa strukturalar vaziyatni saqlaydi. Bunday vaziyatda shaxsning atributlarini strukturaga birlashtiramiz va strukturalar massivini yaratamiz. Bu yerda, standart qiymatlarni o'rnatish mumkin bo'lgan konstruktor ham bo'lishi mumkin:

```
struct Person
{
    public Person(string familiya, string ism)
    {
        Familiya = familiya;
        Ism = ism;
        yosh = 18;
    }
    public string Familiya;
    public string Ism;
    public int yosh;
}
```

Ba'zi strukturalar boshqa strukturalar va sinflardan voris ololmaydi yoki boshqa strukturalar va sinflar uchun asos bo'lib xizmat qiladi (strukturalar C# dagi barcha boshqa ma'lumotlar turlari kabi, obyekt sinfini voris qilib oladi). Biroq strukturada vergul bilan ajratilgan ro'yxatdagi struktura nomidan keyin ko'rsatilgan bir yoki bir nechta interfeyslarni amalga oshirish mumkin. Sinflar singari, har bir strukturaning o'z a'zolari bor: *metodlar*, *maydonlar*, *indeksatorlar*, *xossalar*, *operator* *metodlar* va *hodisalar*. Strukturalarga konstruktorlarni aniqlashga ruxsat beriladi, ammo destruktorlarni emas. Shu bilan birga struktura uchun standart konstruktorni aniqlab bo'lmaydi (ya'ni parametrsiz konstruktor). Chunki standart konstruktor barcha strukturalar uchun avtomatik ravishda aniqlanadi va ularni o'zgartirish mumkin emas. Ushbu konstruktor strukturaning maydonlarini standart qiymatlariga moslashtiradi. Strukturalar

vorislikni qo'llab-quvvatlamagani uchun, ularning a'zolari *abstrakt*, *virtual* yoki *protected* deb belgilanishi mumkin emas.

Struktura obyektini *new* operator yordamida sinf obyektini singari yaratish mumkin, ammo bunga alohida ehtiyoj yo'q. Axir, *new* operator ishlatilganda, standart konstruktor chaqiriladi va bu operator ishlatilmaganda, obyekt ishga tushirilmasa ham, baribir yaratiladi. Bunday holda, strukturaning har qanday a'zolarini qo'lda boshlash kerak bo'ladi.

Quyidagi namunaviy dastur kitob haqidagi ma'lumotni saqlash uchun strukturadan foydalanishni namoyish etadi.

```
using System;
// Strukturani aniqlash
struct Kitob
{
    public string Muallif;
    public string Nomi;
    public int Yili;
    public Kitob(string a, string t, int c)
    {
        Muallif = a;
        Nomi = t;
        Yili = c;
    }
}
// Kitob strukturasiidan foydalanish
class StructDemo
{
    static void Main()
    {
        Kitob kitob1 = new Kitob("Герберт Шилдт", "Полный справочник по C# 4.0",
2010); // aniq konstruktor chaqiruvi
        Kitob kitob2 = new Kitob(); // standart konstruktorni chaqirish
        Kitob kitob3; // konstruktor chaqirilmaydi
        Console.WriteLine(kitob1.Muallif + ", " + kitob1.Nomi + ", (c) " + kitob1.Yili);
        Console.WriteLine();
        if (kitob2.Nomi == null)
            Console.WriteLine("kitob2.nomi bo'sh");
        // Endi kitob2 tarkibiga ma'lumotlarni kiriting.
        kitob2.Nomi = "Язык программирования C# 7 и платформы .NET и .Net Core";
        kitob2.Muallif = "Эндрю Троэлсен, Филипп Джепикс";
        kitob2.Yili = 2018;
        Console.WriteLine("kitob2 strukturasi endi o'z ichiga oladi:\n");
        Console.WriteLine(kitob2.Muallif + ", " + kitob2.Nomi + ", (c) " + kitob2.Yili);
        Console.WriteLine();
        // Console.WriteLine(kitob3.Nomi); // bu a'zo noto'g'ri
        // oldin boshlang'ich qiymat berish kerak
        kitob3.Nomi = "C# 7.0 Справочник";
        kitob3.Muallif = "Джозеф Албахари и Бен Албахари";
        kitob3.Yili = 2018;
        Console.WriteLine(kitob3.Nomi); // endi to'g'ri
        Console.ReadKey();
    }
}
```

Ushbu dastur quyidagi natijani beradi.

Герберт Шилдт, Полный справочник по C# 4.0, (с) 2010

kitob2.nomi bo'sh

kitob2 strukturasi endi o'z ichiga oladi:

Эндрю Троэлсен, Филипп Джепикс, Язык программирования C# 7 и платформы .NET и .Net Core, (с) 2018

C# 7.0 Справочник

Yuqoridagi dasturdan ko'rinib turibdiki, *new* operator bilan konstruktorni chaqirish uchun yoki shunchaki obyektни e'lon qilish orqali strukturani boshlash mumkin. Masalan, agar *new* operator ishlatilsa, strukturaning maydonlari e'lon bo'yicha konstruktor (bu holda barcha maydonlar o'zlarining standart qiymatlariga o'rnatiladi) yoki foydalanuvchi tomonidan belgilangan konstruktor orqali boshlanadi. Agar *new* operator ishlatilmasa, xuddi kitob3 strukturasi bo'lgani kabi, struktura obyektі ishga tushirilmaydi va ushbu obyektни ishlatishdan oldin uning maydonlari qo'lda o'rnatilishi kerak.

Bitta struktura boshqasiga tayinlanganda uning obyektі nusxasi yaratiladi. Bu struktura va sinf o'rtasidagi asosiy farqlardan biridir. Bitta sinfga havola boshqa sinfga havola tayinlanganda, tayinlash operatorining chap qismidagi havola o'ng tomonidagi havola bilan bir xil obyektga ishora qiladi va bitta strukturaning o'zgaruvchisi boshqa strukturaning o'zgaruvchisiga berilganda, tayinlash operatorining o'ng tomonidan struktura obyektining nusxasi yaratiladi. Misol tariqasida quyidagi dasturni ko'rib chiqamiz.

```
using System;
// Strukturani aniqlash
struct MyStruct
{
    public int x;
}
// Strukturani tayinlash
class StructAssignment
{
    static void Main()
    {
        MyStruct a;
        MyStruct b;
        a.x = 10;
        b.x = 20;
        Console.WriteLine("a.x={0}, b.x={1}", a.x, b.x);
        a = b;
        b.x = 30;
        Console.WriteLine("a.x={0}, b.x={1}", a.x, b.x);
        Console.ReadKey();
    }
}
```

Bu ushbu dasturni ishga tushirish natijasidir.

a.x=10, b.x=20

a.x=20, b.x=30

Yuqoridagi natijada ko'rsatilganidek, topshiriqdan keyin *a = b;* *a* va *b* struktura o'zgaruvchilari hali ham butunlay ajralib turadi, ya'ni *a* o'zgaruvchisi *b* o'zgaruvchiga ishora qilmaydi va u bilan hech qanday aloqasi yo'q, faqat *b* o'zgaruvchisi qiymatining nusxasini o'z ichiga oladi. Agar *a* va *b* o'zgaruvchilar ma'lum bir sinf obyektlariga ishora qiluvchi mos havola turida bo'lsa, vaziyat butunlay boshqacha bo'ladi. Misol tariqasida quyida avvalgi dasturning ma'lum bir sinf obyektlariga murojaat qilish uchun o'zgaruvchilar berilishini namoyish etadigan varianti keltirilgan.

// Muayyan sinf obyektlariga havolalardan foydalanish

```

using System;
// Sinf yaratish
class MyClass
{
    public int x;
}
// Berilgan sinfnig turli xil obyektlarini tayinlashni ko'rsatish
class ClassAssignment
{
    static void Main()
    {
        MyClass a = new MyClass();
        MyClass b = new MyClass();
        a.x = 10;
        b.x = 20;
        Console.WriteLine("a.x={0}, b.x={1}", a.x, b.x);
        a = b;
        b.x = 30;
        Console.WriteLine("a.x={0}, b.x={1}", a.x, b.x);
        Console.ReadKey();
    }
}

```

Ushbu dasturning bajarilishi quyidagi natijaga olib keladi.

a.x=10, b.x=20

a.x=30, b.x=30

Ko'rib turganingizdek, a o'zgaruvchiga b o'zgaruvchisi tayinlangandan so'ng har ikkala o'zgaruvchi bir xil obyektga ishora qiladi, ya'ni dastlab b obyektga ishora qilgani kabi.

Strukturalarning maqsadi

C# sinflardan ko'ra oddiyroq imkoniyatlarga ega bo'lsa, strukturani nega o'z ichiga oladi? Bu savolga javob dasturning ish sifati va samaradorligini oshirish hisoblanadi. Strukturalar qiymat turlaridir va shuning uchun mos havola orqali emas, balki to'g'ridan to'g'ri boshqarilishi mumkin. Shunday qilib, struktura bilan ishlash uchun mos havola turidagi o'zgaruvchi umuman talab qilinmaydi, bu ba'zi hollarda operativ xotirani sezilarli darajada tejashni bildiradi. Bundan tashqari, struktura bilan ishlash sinf obyektiga kirish uchun ishlashning pasayishiga olib kelmaydi. Strukturaga kirish to'g'ridan to'g'ri, obyektlarga esa mos havola bo'yicha amalga oshiriladi, chunki sinflar mos havola turidagi ma'lumotlarga tegishli. Agar voris va havola bo'yicha ma'lumotni talab qilmaydigan bir-biriga bog'langan ma'lumotlar guruhini saqlash kerak bo'lsa, unda ishlash nuqtayi nazaridan ular uchun strukturani tanlash yaxshiroqdir.

Quyida strukturaning amalda qanday qo'llanilishini ko'rsatadigan yana bir misol keltirilgan. Ushbu misol elektron tijorat sohasidagi tranzaksiya yozuvini jo'natish. Har bir bunday tranzaksiya paketning soni va uzunligini o'z ichiga olgan paket sarlavhasini o'z ichiga oladi. Sarlavhadan keyin hisob raqami va tranzaksiya summasi ko'rsatiladi. Paket sarlavhasi o'z-o'zidan ma'lumot birligidir va shuning uchun u alohida strukturaga tashkil etilgan bo'lib, undan keyin tranzaksiya yozuvi yoki boshqa turdagi ma'lumotlar paketini yaratish uchun foydalaniladi.

```

using System;
struct PacketHeader
{
    public uint PackNum; // Paket nomeri
    public ushort PackLen; // Paket uzunligi
}
class Transaction
{

```

```

static uint transacNum = 0;
PacketHeader ph; // Transaction sinfiga PacketHeader strukturasini kiritish
string accountNum;
double amount;
public Transaction(string acc, double val)
{
    // paket sarlavhasini yaratish
    ph.PackNum = transacNum++;
    ph.PackLen = 512; // ixtiyoriy uzunlik
    accountNum = acc;
    amount = val;
}
// Transaksiyani jo'natish
public void sendTransaction()
{
    Console.WriteLine("Paket: " + ph.PackNum +
        ", Uzunlik: " + ph.PackLen +
        ",\nNomeri: " + accountNum +
        ", Summa: {0}\n", amount);
}
}
// Tranzaksiya paketi sifatida strukturadan foydalanishni ko'rsatish
class PacketDemo
{
    static void Main()
    {
        Transaction t = new Transaction("31243", 100.12);
        Transaction t2 = new Transaction("AB4655", 345.25);
        Transaction t3 = new Transaction("8475-09", 9800.00);
        t.sendTransaction();
        t2.sendTransaction();
        t3.sendTransaction();
        Console.ReadKey();
    }
}

```

Dastur natijasi:

```

Paket: 0, Uzunlik: 512,
Nomeri: 31243, Summa: 100,12
Paket: 1, Uzunlik: 512,
Nomeri: AB4655, Summa: 345,25
Paket: 2, Uzunlik: 512,
Nomeri: 8475-09, Summa: 9800

```

PacketHeader strukturasini tranzaksiya paketining sarlavhasini shakllantirish uchun juda mos keladi, chunki u juda kam ma'lumotlarni saqlaydi, vorislikdan foydalanmaydi va hatto metodlarni o'z ichiga olmaydi. Bundan tashqari, PacketHeader strukturasini bilan ishlash obyektga havolalar bilan bog'liq qo'shimcha xarajatlarni talab qilmaydi, bu sinf uchun juda xosdir. Shuning uchun PacketHeader strukturasini jarayonning samaradorligini pasaytirmasdan har qanday tranzaksiyani yozib olish uchun ishlatilishi mumkin.

Qizig'i shundaki, C++ tili ham strukturalarga ega va *struct* kalit so'zidan foydalanadi. Ammo bu strukturalar C# tilidagidan farq qiladi. Demak, C++ da struktura sinf turi bo'lib, bu tildagi struktura va sinf amalda ekvivalent bo'lib, bir-biridan faqat sinf uchun shaxsiy bo'lgan va

sinf uchun ochiq bo'lgan a'zolariga ruxsat bo'yicha kirishda farqlanadi. C# da struktura qiymat turiga, sinf esa mos havola turiga ega.